Problemi di ottimizzazione

Materia	Algoritmi
Anno	Secondo Anno
Data	@November 19, 2024
Studiato	Done

In un problema P è possibile trovare numerose soluzioni S. Talvolta cerchiamo di trovare una soluzione migliore delle altre.

Allora date $S_1, S_2, ..., S_n$ soluzioni, introduciamo una **funzione bontà** $F : S^{(P)} \to \mathbb{R}$ che associa un numero ad una soluzione appartenente all'insieme di soluzioni $S^{(P)}$. Allora cerchiamo di **massimizzare o minimizzare il valore di F.**

In alcuni casi si parla di soluzione al problema indicando l'algoritmo che risolve il problema. Ma in realtà questo non è realmente il caso.

> *Un problema di ottimizzazione è un problema computazionale sulle cui variabili sono definite una o più funzioni di costo. L'obiettivo è trovare una soluzione che:*
>
> - *soddisfi i requisiti del problema (**hard constraints**);*
> - *ottimizzi o minimizzi una **funzione obiettivo** condizionata dalla **funzione di costo** (**soft constraints**).*
>
> *Una soluzione che soddisfa i requisiti del problema e minimizza o massimizza la funzione obiettivo è una **soluzione ottima**.*

Programmazione dinamica

La **programmazione dinamica** è una tecnica di risoluzione di un problema di ottimizzazione. Tale problema di ottimizzazione necessita di una **struttura ricorsiva**. Inoltre, deve godere della proprietà di **sottostruttura ottima**.

Tra le tante soluzioni S_i, cerchiamo la soluzione **ottima**. Indichiamo tale soluzione con S^*. Se S^* è stata ottenuta unendo soluzioni S_i a sottoproblemi P_i, allora le soluzioni S_i sono **soluzioni ottime** S_i^*. Questo perché ogni sottoproblema ha molte soluzioni, e per ottenere una soluzione finale ottima, anche le singole soluzioni devono essere ottime. Questa è la proprietà di sottostruttura ottima.

Dimostrare che un problema gode della proprietà di sottostruttura ottima, richiede una dimostrazione per contraddizione (per assurdo).

Passi per risolvere un problema con la programmazione dinamica

Il primo passo è costruire una funzione ricorsiva che calcola il costo di una soluzione ottima. Poiché la funzione ricorsiva ricalcola alcuni valori più volte, invece di risolvere il problema tramite memorizzazione, risolviamo il problema con un **approccio bottom-up**.

Il problema che risolveremo è il problema di **rod-cutting**. Supponiamo di avere un'azienda che taglia barre di lunghezza n in pezzi più piccoli. Supponiamo di avere la tabella di prezzi:

n	1	2	3	4	5	6	7
prezzo p	1	5	8	9	10	17	17

L'azienda sa quanto viene pagata la barra di lunghezza n. Ci chiediamo **qual è il modo migliore di tagliare la barra per massimizzare il guadagno**.

Definiamo la funzione $r(n)$ ricorsiva:

$$r(n) = \begin{cases} 0 & n = 0 \\ \max\{\, P(i) + r(n-i) \text{ per } 1 \leq i \leq n \,\} & n > 0 \end{cases}$$

Quindi in pseudocodice:

```
ROD-CUTTING(n, p):
   if n = 0 then
      return 0
   q = - inf
   for i = 1 to n do
      r = p(i) + ROD-CUTTING(n-i, p)
      if (q < r) then
         q = r
   return q
```

Chiaramente se osserviamo l'albero di ricorsione, ripetiamo il calcolo di alcuni nodi. Quindi utilizziamo la memorizzazione:

```
ROD-CUTTING(n, p):
   r = new-array(n+1)
   for i = 0 to n do
      r[i] = -inf
   r[0] = 0
   return ROD-CUTTING-AUX(n,p)

ROD-CUTTING-AUX(n, p):
   if n = 0 then
      return 0
   q = - inf
   for i = 1 to n do
      if r[n-1] = -inf then
         r[n-1] = ROD-CUTTING-AUX(n-i, p)
      if (q < p(i) + r[n-i]) then
         q = p(i) + r[n-i]
   return q
```

Questa procedura ha complessità quadratica.

Cerchiamo ora di risolvere il problema con la programmazione dinamica utilizzando una metodologia **bottom-up**:

```
ROD-CUTTING(n, p):
   r = new-array(n+1)
   r[0] = 0
   for k = 1 to n do:
      r[k] = -inf
      for i = 1 to k do
         if r[k] < P(i) + r[k-i] then
            r[k] = P(i) + r[k-i]
   return r[n]
```

Nota: p è un array indicizzato $[1 : n]$, r è indicizzato $[0 : n]$.

Questo ci permette di calcolare il guadagno massimo, ma **non sappiamo quali tagli fare**. Aggiungiamo allora un array T che tiene conto dei tagli ad ogni iterazione:

```
ROD-CUTTING(n, p):
   r = new-array(n+1)
   t = new-array(n+1)
   r[0] = 0
   for k = 1 to n do:
      r[k] = -inf
      for i = 1 to k do
         if r[k] < P(i) + r[k-i] then
            r[k] = P(i) + r[k-i]
            t[k] = i
   return r, t
```

E per stampare la sequenza di tagli:

```
PrintCuts(n, t):
   while n > 0 do
      print t[n]
      n = n - t[n]

# ricorsivo:
PrintCuts(n, t):
   if n = 0 then return
   print t[n]
   printCuts(n - t[n], t)
```

Esempio

Sia data una barra di lunghezza $n = 4$ e l'array di prezzi $P = [1, 5, 8, 9, 10, 17]$. Allora inizializziamo i nostri array T:

$$r = [0, -\infty, -\infty, -\infty, -\infty, -\infty, -\infty,]$$
$$t = [N, N, N, N, N, N, N]$$

Allochiamo spazi in più per successivi esempi. Per ogni iterazione di k si ha:

$$r = [0, 1, -\infty, -\infty, -\infty, -\infty, -\infty]$$
$$t = [0, 1, N, N, N, N, N]$$
$$\downarrow$$
$$r = [0, 1, 5, -\infty, -\infty, -\infty, -\infty]$$
$$t = [0, 1, 2, N, N, N, N]$$
$$\downarrow$$
$$r = [0, 1, 5, 8, -\infty, -\infty, -\infty]$$
$$t = [0, 1, 2, 3, N, N, N]$$
$$\downarrow$$
$$r = [0, 1, 5, 8, 10, -\infty, -\infty]$$
$$t = [0, 1, 2, 3, 2, N, N]$$

Quindi la nostra soluzione è $r[4] = 10$.

Ora per esempio per trovare la soluzione con $n = 5$ basta che analizziamo il valore di $r[4]$ e il prezzo $P(i)$ per $i = 1 \to 5$, e di conseguenza la soluzione la troviamo immediatamente:

$$r = [0, 1, 5, 8, 10, -\infty, -\infty]$$
$$t = [0, 1, 2, 3, 2, N, N]$$
$$\downarrow$$
$$r = [0, 1, 5, 8, 10, 13, -\infty]$$
$$t = [0, 1, 2, 3, 2, 2, N]$$